Q2:Prove L is in NP 1.You are required to prove that the following problem L is in NP. To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of pairs in the form (G, k) where G is an undirected graph and k is a natural number such that there is a subset S of the set of edges of G having cardinality at most k and such that any node of G occurs in at least one of the edges in S.

Certificate S is the subset of edges

```
Verfier(G(V,E),k,S):
If |S|>k:
return 0
found=false
For vertex E V:
  For (u,v)  in S:
   If u E vertex or v E vertex:
     found=true
     Break
  If found=flase:
    Return 0

Return 1
```

Q2 You are required to prove that the following problem L is in N P. To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of pairs in the form (G, n) where G is an undirected graph having a path which goes through n distinct edges of G, going through each of them exactly once. Can you say anything else about the complexity of this problem?

```
CERTIFCATE  L
Verifier (G(V,E) ,n)

Visited = {}
For (u,v) E L:
 If u !E  L or v!E L:
  Return 0

For u in L:
 If u in visitted:
 return 0
Else
 Visited = visited U {u}
return 1
```

You are required to prove that the following problem L is in N P. To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of pairs in the form (G, k) where G is a graph and k is a natural number, such that the nodes of G can be assigned a natural number between 1 and k in such a way that nodes which are linked by an edge are assigned distinct natural numbers
Certificate C
Verifier(G(V,E),k)

```
 For each vertex v in V:
       If C[v] < 1 or C[v] > k:
          Return 0  // Reject: Invalid color assigned

    // Step 2: Verify that no two adjacent vertices share the same color
    For each edge (u, v) in E:
       If C[u] == C[v]:
          Return 0  // Reject: Adjacent vertices have the same color

    // Step 3: If all checks pass, the coloring is valid
    Return 1  // Accept
```

You are required to prove that the following problem L is in NP. To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of pairs in the form (G, t) where G is a graph and t is a natural number, such that the nodes of G can be partitioned into at most t sets in such a way that nodes which are part of distinct sets are not linked by any edge. ...or equivalentl

Certificate L is t sets
Verifier(G(V,E),t):

```
For i <-1 to t:
  For j< 1 to -t:
   If (i !=j): //distinct sets

     If there is V E Vi  ,W E Vj:
      If {V,W} E edge
        return false
Return true
```

You are required to prove that the following problem L is in NP . To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of triples in the form (n, m, k) where n and m are natural numbers which have at least k prime factors in common.

```
Verifier(n, m, k, Certificate P):
    // P is a list of candidate prime factors: [p_1, p_2, ..., p_k]

    // Step 1: Verify the certificate contains exactly k numbers
    If |P| != k:
        Return 0

    // Step 2: Verify each number is a valid common prime factor
    For each p in P:
        // Check if p is a prime number
        // (Note: Primality testing is in P via the AKS algorithm)
        If IsPrime(p) == False:
            Return 0

        // Check if p divides n
        If n mod p != 0:
            Return 0

        // Check if p divides m
        If m mod p != 0:
            Return 0

    // Step 3: Verify the prime factors are distinct
    // (Standard interpretation of "prime factors" implies distinct divisors)
    If P contains duplicate values:
        Return 0

    // Step 4: All checks passed, the numbers share at least k prime factors
    Return 1
```

You are required to prove that the following problem is L in NP. To do that, you proceed as you prefer (e.g. by giving a NTM, or by giving certificates and algorithms). The problem is one that, given a natural number n and a set of natural numbers D, checks whether n is a product of distinct elements from D

```
Verifier(n, D, Certificate S):
    // S is a list of candidate numbers: [x_1, x_2, ..., x_k]

    // Step 1: Verify that S contains only distinct elements from D
    If S contains duplicate values:
        Return 0  // Reject: Elements must be distinct

    For each x in S:
        If x is not in D:
            Return 0  // Reject: Element is not from the set D

    // Step 2: Compute the product of all elements in S
    product = 1
    For each x in S:
        product = product * x

    // Step 3: Verify if the product equals n
    If product == n:
        Return 1  // Accept
    Else:
        Return 0  // Reject
```

9. You are required to prove that the following problem L is in NP. To do that, you can give a N TM or define some pseudocode. The problem is one that, given a natural number n, checks whether n is a sum of powers of 3. (3^0 is not allowed as the problem would be trivial)

```
Verifier( n,C)
Certificate is list of exponents
For e in C:
If e<1:
Return 0
sum=0
For e in C:
 co=computePower(3,e)
 sum+=sum+co
If n == co:
 Return 1
```

Else
Return 0

You are required to prove that the following problem L is in N P. To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of pairs of strings (t, z) both of them from the alphabet S= {0,1,2...,9} such that s is an anagram of t, i.e., the former can be obtained from the latter by changing the order of the letters

```
EasyVerifier(t, z, EmptyCertificate):
    If |t| != |z|:
        Return 0

    // Sort both strings
    sorted_t = Sort(t)
    sorted_z = Sort(z)

    // If they are anagrams, their sorted versions will be identical
    If sorted_t == sorted_z:
        Return 1
    Else:
        Return 0
```

You are required to prove that the following problem L is in NP. To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of triples in the form (G, n, m) where G is an undirected graph, n, m are natural numbers, and G contains two independent sets of size at least equal to n and m, respectively. (Two sets are disjoint if their intersection is empty.) Can you say anything else about the complexity of O?)

Independent set is tgat there should be no edge between the vertices

Check if $S_1 \subseteq V$ and $S_2 \subseteq V$.
If not, RETURN 0.

. Check if $|S_1| \geq n$ and $|S_2| \geq m$.
If not, RETURN 0.

Check if $S_1 \cap S_2 = \varnothing$ (i.e., they are disjoint).
If not, RETURN 0.

. Check if S_1 is an independent set:
For every pair of vertices (u, v) in S_1:

If (u, v) ∈ E
RETURN 0.

Check if S_2 is an independent set:
For every pair of vertices (u, v) in S_2:
If (u, v) ∈ E,
RETURN 0.

RETURN 1,.

. You are required to prove the function f is in FP. To do that, you can give a TM or define some pseudocode. The function is one that given two lists L= L1...Ln and P= P1....Pn of rational numbers, returns their scalar product

Function ScalarProduct(L, P)

Input:
Two lists L = (L1, L2, ..., Ln)
P = (P1, P2, ..., Pn)
where each Li and Pi is a rational number

Output:
The scalar product:
L1P1 + L2P2 + ... + LnPn

Algorithm:

1. If length(L) ≠ length(P), reject.

2. sum ← 0

3. For i = 1 to n do:
product ← Li × Pi
sum ← sum + product

4. Return sum

You are required to prove that the following function f is in FP. To do that, you can give a TMs or define some pseudocode. The function is one that, given two strings v, w ∈ S* checks whether v is a substring of w, namely whether w contains an exact occurrence of v.

Function IsSubstring(v,w)

1. m ← length(v)
2. n ← length(w)

3. For i = 1 to n-m+1

    match ← true

    For j = 1 to m

        If v[j] ≠ w[i+j-1]
            match ← false
            break

    If match = true
        return 1

4. return 0

2. You are required to prove that the following problem L is in NP. To do that, you can give a TM or define some pseudocode. The language L includes precisely those binary strings which are encodings of natural numbers which can be factored into 2 distinct prime numbers. In doing so you can assume that primality can be tested in polynomial time. Examples of numbers which are in L are 15=3·5 or 35=5·7.

Input X
Certificate p,q

Verifier
If prime(p) and prime(q):
  If x=p*q:
  Return True
  else:
    Return False